4. Operating Systems

Operating Systems are system software that act as an intermediary between computer hardware and users or application programs. They manage hardware resources and provide essential services that enable efficient and secure system operation.

The primary purposes of an **Operating System** are to enable applications (softwares) to interact with a computer's hardware and to manage a system's hardware and software resources.

a. Resource Management

Resource management is one of the core responsibilities of an Operating System. It ensures that hardware components such as the CPU, memory, storage, and input or output devices are used efficiently, fairly, and securely. Without proper management, multiple programs running at the same time would conflict with each other, causing crashes, slow performance, or data loss.

The OS is responsible for allocating and controlling system resources to ensure optimal performance:

- **CPU Management**

  CPU management involves deciding which process gets access to the processor and for how long. Since only one process can use a CPU core at a time, the Operating System uses scheduling algorithms to distribute CPU time.

  The OS Performs the following tasks:
  - Creates and deletes processes
  - Schedules processes using algorithms such as First Come First Served (FCFS), Shortest Job First, Priority Scheduling, and Round Robin.
  - Performs context switching
  - Handles multitasking.

  Example:
  In Microsoft Windows, when you open a browser, a music player, and a code editor at the same time, the OS rapidly switches between these processes so they appear to run simultaneously.

  Why is it important?

    - Prevents CPU starvation
    - Improves system responsiveness
    - Enables multitasking

- **Memory Management**
  Memory management controls and coordinates computer memory, assigning portions to programs while preventing conflicts. The OS ensures that each process has enough memory to execute while protecting other processes from unauthorized access.

  The OS performs:
    - Allocation and deallocation of RAM
    - Virtual memory management
    - Paging and segmentation
    - Memory protection

  Example:
  If your laptop has 8GB RAM and you open multiple applications, the OS uses virtual memory to move inactive data to disk storage, allowing active programs to continue running.

  Why is it important:
    - Prevents memory conflicts
    - Improves system stability
    - Allows large applications to run even with limited RAM

- **Storage Management (File System Management)**
  Storage management handles data organization, file systems, and disk space allocation. The OS ensures that data is stored efficiently and can be retrieved quickly.

  The OS performs:
    - File creation and deletion
    - Directory management
    - Disk space allocation
    - Access control and permissions

  Example:
  When you save a document, the OS determines where on the disk it should be stored and records its location in the file system.

Why is it important:

- Organizes data systematically
- Prevents data corruption
- Ensures secure file access

- **Device Management**
  Device management controls communication between the computer and hardware devices such as keyboards, printers, monitors, and storage drives.

  The OS performs:
  - Device driver management
  - Buffering and caching
  - Interrupt handling
  - I/O scheduling

  A device driver acts as a translator between the OS and hardware devices.

  Example:
  When you connect a printer, Microsoft Windows automatically installs a driver so the system can communicate with the printer.

  Why is it important:
  - Enables hardware compatibility
  - Improves I/O performance
  - Ensures efficient communication between software and hardware.

In summary, the resource management allows the operating system to allocate CPU time fairly among processes, manage memory efficiently using virtual memory techniques, organize and secure data using file systems, and control hardware devices through drivers and I/O systems. Through proper resource management, the OS ensures system efficiency, reliability, multitasking capability, and security.

b. Process Scheduling and Multitasking

Process scheduling is a core function of the Operating System that enables multitasking. Multitasking allows multiple programs to appear as if they are running at the same time, even on a single CPU. This is possible because the OS rapidly switches the CPU between active processes.

The OS supports multitasking, allowing multiple programs to run seemingly at the same time:

- Uses **scheduling algorithms**
  The Operating System uses scheduling algorithms to determine which process runs next. These algorithms aim to balance fairness, efficiency, and responsiveness. Common scheduling methods include:
  - **First Come First Served (FCFS)**
    In First Come, First Served scheduling, the process that arrives first in the ready queue is the first one to be assigned to the CPU. If two processes arrive at the same time, the one with the smaller process ID is executed first. This scheduling method follows a strict first come, first served basis and is non preemptive in nature, meaning that once a process starts executing, it continues until it finishes or voluntarily releases the CPU. Its implementation is based on the FIFO queue structure, which makes it simple and easy to understand and implement.

    Advantages:
    - It is simple and easy to understand.
    - It can be easily implemented using queue data structure.
    - It does not lead to starvation.

    Disadvantages:
    - It does not consider the priority or burst time of the processes.
    - It suffers from convoy effect i.e. processes with higher burst time arrived before the processes with smaller burst time.

  - **Shortest Job First (SJF)**
    In Shortest Job First, the process with the shortest CPU burst time is executed first. If two processes have the same burst time, the First Come, First Served method is used to decide which one runs first. This scheduling approach can be either non preemptive or preemptive. It is considered one of the best methods for minimizing average waiting time, especially in batch systems where the required CPU time of each process is known beforehand. However, it is difficult to apply in interactive systems because the exact burst

time of processes is usually not known in advance. For SJF to work properly, the processor must know how long each process will take. The preemptive version of SJF is called Shortest Remaining Time First, where the process with the smallest remaining execution time is always selected.

Advantages:

- SRTF is optimal and guarantees the minimum average waiting time.
- It provides a standard for other algorithms since no other algorithm performs better than it.

Disadvantages:

- It can not be implemented practically since the burst time of the processes can not be known in advance.
- It leads to starvation for processes with larger burst time.
- Priorities can not be set for the processes.
- Processes with larger burst time have poor response time.

○ **Priority Scheduling**

In Priority Scheduling, the CPU is assigned to the process with the highest priority among all available processes. If two or more processes have the same priority, the tie is resolved using First Come, First Served scheduling. This algorithm can operate in either preemptive or non preemptive mode. In preemptive mode, the process with the highest priority will always have zero waiting time because it can immediately take control of the CPU when it arrives. In non preemptive mode, however, the highest priority process may still have to wait if another process is currently executing. Priority scheduling behaves the same in both modes when all processes arrive at the same time and become available simultaneously.

Advantages:

- It considers the priority of the processes and allows the important processes to run first.
- Priority scheduling in pre-emptive mode is best suited for real time operating systems.

Disadvantages:

- Processes with lesser priority may starve for CPU.

    - There is no idea of response time and waiting time.

  - **Round Robin**
    In Round Robin Scheduling, the CPU is assigned to processes in the order they arrive, following the First Come, First Served principle, but only for a fixed amount of time known as the time quantum or time slice. Once a process uses up its time quantum, it is preempted and placed back into the ready queue, and the CPU is then given to the next process in line. Because processes are regularly interrupted after each time slice, Round Robin is always preemptive in nature.

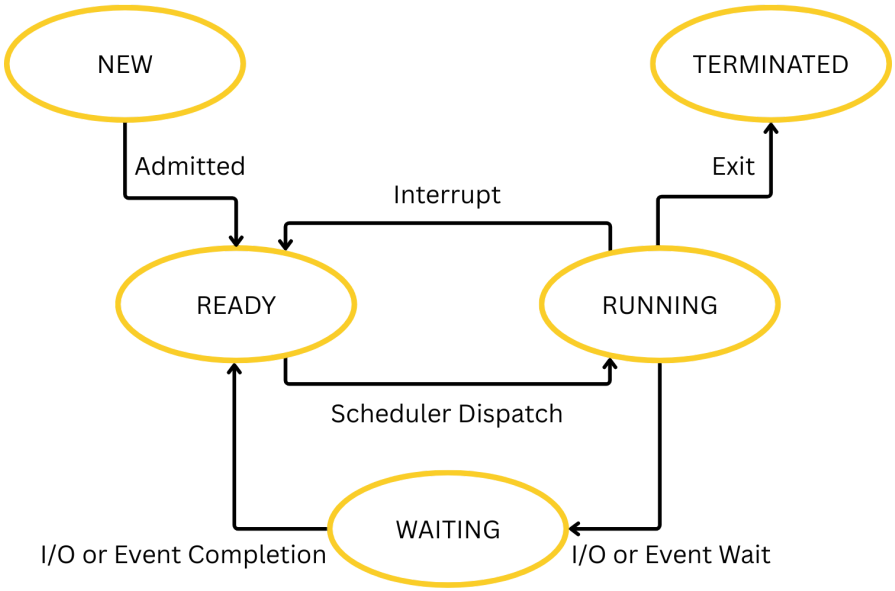
    Advantages:
    - It gives the best performance in terms of average response time.
    - It is best suited for time sharing system, client server architecture and interactive system

    Disadvantages:
    - It leads to starvation for processes with larger burst time as they have to repeat the cycle many times.
    - Its performance heavily depends on the time quantum.
    - Priorities can not be set for the processes.

    When the time quantum is small, the number of context switches increases, response time improves, and the chances of starvation decrease. On the other hand, when the time quantum is large, the number of context switches decreases, response time becomes slower, and the chances of starvation increase. As the time quantum continues to increase, Round Robin gradually behaves like First Come, First Served scheduling, and if the time quantum approaches infinity, it effectively becomes FCFS. Therefore, the time quantum should be selected carefully so that it is neither too large nor too small in order to balance response time and context switching overhead.

- Manages **process states**

Process State



A process goes through different states from the time it is created until it finishes execution. These states describe the current condition of the process and help the operating system manage CPU scheduling and resource allocation properly.

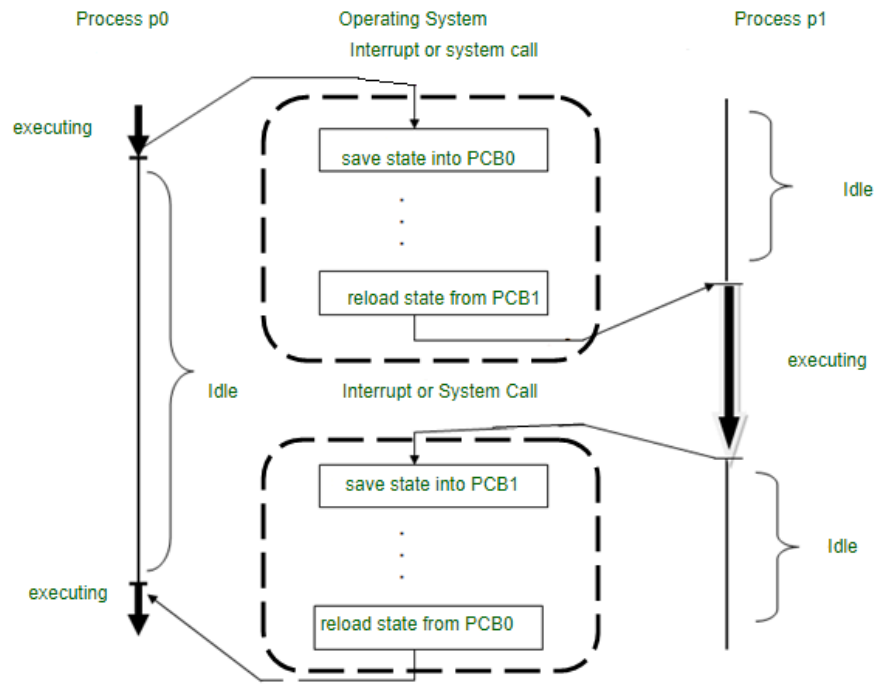
Each process may be in one of the following states:

- NEW - the process is being created.
- RUNNING - instructions are being executed.
- WAITING - the process is waiting for some event to occur (such as an I/O completion or reception of a signal).
- READY - the process is waiting to be assigned to a processor.
- TERMINATED - the process has finished execution.

A process begins in the NEW state when it is being created. At this state, the operating system prepares the necessary data structures, such as the Process Control Block. Once the process is ready to run and waiting for CPU time, it enters the READY state. When the scheduler selects it and assigns the CPU, the process moves to the RUNNING state, where its instructions are actively executed.

If the running process needs to wait for an event such as input or output completion, it transitions to the WAITING state. It remains there until the required event occurs. After the event is completed, the process returns to the READY state to wait for CPU time again. Finally, once the process finishes execution, it enters the TERMINATED state, where the operating system releases its resources.

- Enables **context switching** to quickly switch the CPU between processes.



Context switching is a fundamental operation in an operating system where the CPU changes from executing one process to executing another. Since a processor can handle only one instruction stream at a time, the operating system must frequently switch between tasks to support multitasking. This switch usually happens because of interrupts, which are internal or external signals that require immediate CPU attention, or when a higher priority process becomes ready to run and must take control of the CPU. These operations are common in general purpose systems where many applications run simultaneously.

This ensures responsiveness and fairness among users and applications.

c. *User-System Interface*

The OS provides interfaces through which users interact with the computer:

- **Command-Line Interface (CLI)**

  A Command Line Interface, commonly known as CLI, is a text based user interface that allows users to interact with the operating system by typing commands. Instead of clicking icons or buttons, users enter specific instructions through a keyboard. Examples include the Command Prompt in Windows Command Prompt, the Terminal in Linux distributions, and the Terminal application in macOS.

  Characteristics

  A CLI is text based and relies entirely on typed commands. Users must know the correct syntax and structure of commands to execute tasks properly. It provides direct communication between the user and the operating system, often through a shell program. CLIs are typically lightweight and consume very little system memory compared to graphical interfaces. They also support scripting, which allows users to automate repetitive tasks by writing command sequences in script files. Additionally, CLIs are highly precise, meaning even small typing errors can cause commands to fail.

  Advantages

  One major advantage of a CLI is speed and efficiency for experienced users. Tasks such as file management, program execution, and system configuration can be performed quickly with a few typed commands. It also consumes fewer system resources, making it suitable for low performance machines or servers. CLIs provide powerful control over system operations, allowing advanced users and administrators to perform complex configurations. Another advantage is automation, since scripts can execute multiple commands automatically without manual interaction. CLIs are also ideal for remote system management because they work well over network connections.

  Limitations

  A CLI can be difficult for beginners because it requires memorizing commands and understanding syntax rules. There is no visual guidance such as icons or menus, which makes it less intuitive compared to graphical interfaces. Typing errors can easily lead to incorrect results or system issues. In addition, it is not suitable for tasks that require heavy

graphical interaction, such as image editing or design work. Because of these limitations, many modern systems combine CLI and graphical interfaces to provide flexibility and ease of use.

- **Graphical User Interface (GUI)**
  A Graphical User Interface, commonly known as GUI, is a type of user interface that allows users to interact with a computer system through visual elements such as windows, icons, menus, buttons, and pointers. Instead of typing commands, users perform actions by clicking, dragging, and selecting items on the screen. Examples of operating systems that use GUI include Windows 11, macOS, and Ubuntu, a popular distribution of Linux.

  Characteristics
  A GUI is visually oriented and user friendly, using graphical elements to represent files, folders, and system functions. It follows the WIMP model, which stands for windows, icons, menus, and pointer. Users interact with the system using devices such as a mouse, touchpad, or touchscreen. GUIs provide immediate visual feedback, making it easier to understand actions and system responses. They also support multitasking by allowing multiple windows to be open and managed at the same time. Compared to text based interfaces, GUIs require more system resources such as memory and processing power.

  Advantages
  One of the main advantages of a GUI is ease of use, especially for beginners. Users do not need to memorize commands because options are displayed visually. It reduces the chances of errors since users can select from menus instead of typing precise syntax. GUIs are ideal for tasks that involve graphics, multimedia, and design because they allow direct visual manipulation of objects. They also improve productivity for general users by making navigation and file management more intuitive.

  Limitations
  A GUI consumes more system resources compared to a command line interface, which may reduce performance on low specification systems. It can be slower for advanced users who can execute tasks faster through typed commands. Some complex system configurations may not be easily accessible through graphical menus and may still require command line

tools. In addition, GUIs depend heavily on graphical components, so if the graphical environment fails, system access may become limited.

- **Importance of a Good User Interface**

  Improved Usability
  A well designed interface makes systems easier to learn and operate. Clear navigation, consistent layout, and understandable commands reduce confusion and help users complete tasks efficiently. When users can easily understand how to interact with a system, productivity increases and errors decrease. Good usability also shortens the learning curve, especially for beginners.

  Enhanced Accessibility
  A good user interface ensures that systems are usable by a wide range of people, including those with disabilities. Features such as readable fonts, proper color contrast, screen reader compatibility, and keyboard navigation support make technology more inclusive. Accessibility improves user satisfaction and ensures that more individuals can effectively use the system.

  Better System Control
  An effective interface provides users with clear feedback and control over system operations. It shows system status, confirms user actions, and prevents accidental errors through warnings or confirmation messages. When users feel in control and understand what the system is doing, they are more confident and efficient in performing tasks.

  Increased Productivity
  A good interface streamlines workflows and reduces the number of steps required to accomplish tasks. Logical arrangement of tools, shortcuts, and automation features allow users to work faster. In professional environments, this leads to improved performance and time savings.

  Reduced Errors and Frustrations
  Poorly designed interfaces can cause mistakes, delays, and frustration. A good user interface minimizes these issues by guiding users clearly and providing helpful feedback. Error prevention mechanisms, such as input validation and clear instructions, contribute to smoother system operation.